DAMIL-DCIM+: Automated Dataflow-Aware Layout Synthesis for Digital CIM With Self-Assembled Bitcell Units and MILP-Based Optimization

Chuyu Wang, Xinglong Yan, Zecheng Xu, Ke Hu, Keren Zhu, *Member, IEEE*, Shuo Li, *Member, IEEE*, Fan Yang, *Member, IEEE*, and Xuan Zeng, *Senior Member, IEEE*

***Abstract*— Digital computing-in-memory (DCIM) systems integrate complex digital logic with parasitic-sensitive bitcell arrays, presenting unique physical design challenges. Conventional design strategies often fall short in these systems due to irregular dataflow patterns and excessive interconnect lengths, which degrade performance and increase parasitic effects. As a result, current DCIM implementations frequently rely on manual layout, which is both time-consuming and a major bottleneck in the design cycle. While existing DCIM layout synthesis frameworks attempt to automate this process using template-based placement methods inspired by manual design, their rigid constraints can lead to inefficient area utilization and increased core sizes. To address these limitations, we propose DAMIL-DCIM+, a novel placement framework that combines the structural clarity of template-based methods with the flexibility of optimization-based techniques. Specifically, DAMIL-DCIM+ employs a global dataflow-aware floorplan to guide placement and leverages MILP-based detailed placement to optimize wirelength and preserve dataflow regularity. Inspired by self-assembling design principles, this approach enables scalable and structured integration of parasitic-sensitive components. The hybrid methodology of DAMIL-DCIM+ reduces total wirelength, lowers parasitic effects, and enhances performance while maintaining design regularity. Experimental results on a 28-nm DCIM circuit demonstrate that DAMIL-DCIM+ improves operating frequency by 25.2% and reduces power consumption by 19.6% compared to Cadence Innovus, without increasing core area.**

***Index Terms*— Digital computing-in-memory (DCIM), electronic design automation, physical design.**

I. Introduction

THE convergence of rapid advancements in artificial intelligence (AI) and the proliferation of edge computing has imposed stringent constraints on the power efficiency and performance of edge devices. These systems are increasingly tasked with executing computationally intensive workloads, such as neural networks and natural language processing, which in turn place hard requirements on the throughput and latency of on-chip memory subsystems. Conventional von Neumann architectures, characterized by the physical separation of processing and memory units, are fundamentally ill-equipped to meet these demands due to the substantial energy and latency costs associated with data movement. Consequently, a paradigm shift toward novel computing architectures and their corresponding design methodologies is imperative to efficiently support complex AI workloads at the edge while adhering to strict power and latency constraints.

Among emerging solutions, computing-in-memory (CIM) has gained substantial attention. In particular, SRAM-based digital CIM (DCIM) offers high adaptability and zero accuracy degradation, making it a strong candidate for edge AI applications [1]. Compared to analog CIM (ACIM), which achieves efficiency by exploiting the analog behavior of memory cells, DCIM provides deterministic outputs and better resilience to process variations and noise. Its foundation in digital logic ensures stable performance and simplifies integration with standard CMOS design flows.

DCIM circuits present a unique physical design challenge by integrating digital logic directly within parasitic-sensitive bitcell arrays. Similar to SRAM, the operational integrity of these arrays is highly dependent on the uniformity of bitline and wordline parasitics. Any deviation from a regular, grid-like layout can introduce parasitic variations, leading to increased access delay, higher power consumption, or functional failure. Consequently, preserving the structural regularity of the bitcell array is a critical placement constraint.

This hybrid structure creates a tooling gap in conventional EDA flows. Standard digital place-and-route (P&R) tools, which are optimized for random logic, disrupt the required array regularity and fail to manage critical parasitics effectively. Conversely, memory compilers such as OpenRAM [2] are designed to generate standalone memory blocks and cannot synthesize the integrated computational logic

Received 18 April 2025; revised 30 July 2025; accepted 31 August 2025. Date of publication 9 September 2025; date of current version 23 March 2026. This work was supported in part by the National Natural Science Foundation of China (NSFC) Research Projects under Grant 92473207, Grant 62504054, Grant 92373207, Grant 62304052, Grant 62474051, and Grant 62474050; in part by the Science and Technology Commission of Shanghai Municipality Project under Grant 24JD1400800; and in part by the State Key Laboratory of Integrated Chips and Systems, Fudan University under Grant SKLICS-Z202404. This article was recommended by Associate Editor Y. Lin. *(Chuyu Wang, Xinglong Yan, and Zecheng Xu contributed equally to this work.) (Corresponding authors: Keren Zhu; Xuan Zeng.)*

The authors are with the School of Microelectronics, State Key Laboratory of Integrated Chips and System, Fudan University, Shanghai 200433, China (e-mail: krzhu@fudan.edu.cn; xzeng@fudan.edu.cn).

Digital Object Identifier 10.1109/TCAD.2025.3608064

1937-4151 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:23 UTC from IEEE Xplore. Restrictions apply.

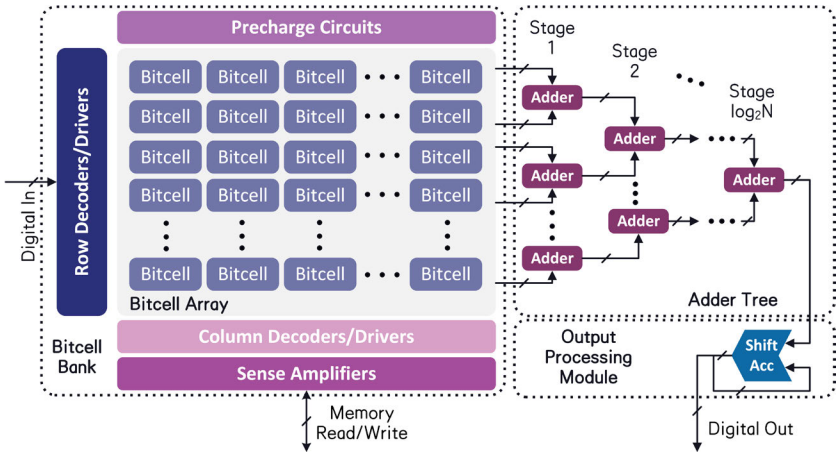


Fig. 1. Typical DCIM circuit architecture.

inherent to DCIM. This lack of automation mandates that DCIM circuits are typically implemented using full-custom, manual layout. While this approach can yield optimized results, it is a labor-intensive and non-scalable process that constitutes a significant bottleneck in the design cycle, motivating the work on an automated placement methodology.

Existing approaches for structured layout synthesis can be broadly categorized into two classes. Template-based methods, employed by tools like AutoDCIM [3] and common in analog design [4], [5], enforce structural regularity through predefined scripts. While effective at preserving critical array structures, their inherent rigidity often results in area overhead and limits portability across different architectures or technology nodes. Alternatively, datapath-aware placement techniques integrate regularity constraints directly into optimization algorithms, enhancing flexibility and area efficiency for digital accelerators [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16]. However, these methods remain suboptimal for managing the stringent, grid-like parasitic constraints of DCIM bitcell arrays.

This analysis above suggests that an effective placement methodology for DCIM must reconcile the conflicting demands of structural rigidity and optimization flexibility. We posit that a hierarchical approach, which synergizes coarse-grained, template-guided global placement with fine-grained, optimization-based local refinement, offers a promising solution. The global template enforces the essential regularity of the bitcell array and its peripheral logic, preserving critical dataflow paths and ensuring uniform parasitic loading on bitlines and wordlines. Concurrently, local optimization algorithms can refine cell placement within these templated regions. This fine-grained refinement addresses objectives such as wirelength minimization and timing optimization for specific critical nets without disrupting the overarching structural integrity.

In this work, we introduce DAMIL-DCIM+, a hierarchical placement framework that automates DCIM layout by bridging template-based global floorplanning with optimization-based detailed placement. The framework first constructs a global floorplan using dataflow-aware templates that enforce the structural regularity required by parasitic-sensitive bitcell arrays. Subsequently, it employs a mixed-integer linear programming (MILP) formulation to perform detailed placement for local logic modules, minimizing area and wirelength while adhering to critical placement constraints. By combining the strengths of template-based placement techniques with dataflow-aware physical design methods, the framework efficiently manages parasitic-sensitive paths while enhancing the circuit's power, performance, and area (PPA) metrics. Our main contributions are summarized as follows.

1) We present DAMIL-DCIM+, a novel DCIM placement framework that simultaneously handles parasitic-sensitive bitcell arrays and digital components, achieving high-quality layout placement.
2) A global dataflow-aware floorplan template is introduced, leveraging the unique intrinsic tree-like dataflow of DCIM circuits to provide overall guidance for the entire layout.
3) An MILP-based detailed placement method is employed to further minimize the local wirelength, reduce the overall area while maintaining local regularity.
4) Experimental results show that our framework reduces total wire length by 37.4%, improves frequency by 25.2%, and decreases power consumption by 19.6% compared to layouts generated by commercial tools, all while maintaining the same core area. Post-layout simulations confirm effective management of parasitic-sensitive paths, ensuring successful bitcell read and write operations.

II. Preliminaries

A. Typical DCIM Architecture

A typical DCIM circuit, as illustrated in Fig. 1, consists of three primary components: the bitcell bank, the adder tree, and the output processing module.

1) Bitcell Bank: The bitcell serves as the fundamental in-memory computation unit, typically integrating a one-bit storage cell with a multiplication device. Due to their high-speed access and low-power characteristics, SRAM-based bitcells are adopted as the core building blocks in most DCIM architectures. These arrays are supported by peripheral circuits and control logic, including address decoders, bitline precharge units, and sense amplifiers. This periphery, which

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:23 UTC from IEEE Xplore. Restrictions apply.

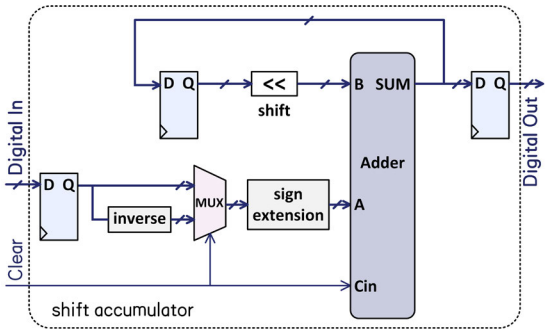


Fig. 2. Classic architecture of the shift accumulator.

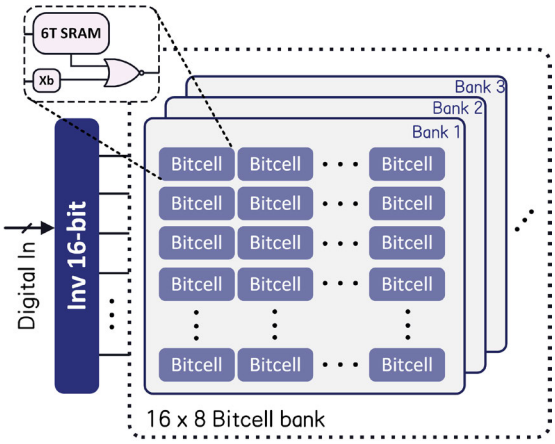


Fig. 3. 10T bitcell array.

closely resembles that of conventional SRAM, ensures precise timing for write operations and accurate data readout. In a traditional SRAM layout, placement is typically performed manually to preserve structural regularity, which improves pattern matching, reduces parasitics, and enhances circuit reliability. Although bitcells occupy only a small portion of the overall DCIM layout, their structural constraints are incompatible with standard digital automation tools due to their specialized placement requirements.

2) Adder Tree: Typical DCIM architectures perform multiply-accumulate (MAC) operations by processing multiple multibit serial data streams in parallel. Each row of the bitcell array acts as an independent input branch, with the bit-width of the serial data determining the number of required clock cycles. At each cycle, DCIM computes partial sums that are accumulated by an adder tree organized as a $\log_2 N$-stage binary structure, where N denotes the number of input rows. This hierarchical accumulation significantly reduces computational latency and improves throughput. However, the adder tree occupies a substantial portion of the layout and hosts timing-critical paths, making its physical placement a key challenge in DCIM design.

3) Output Processing Module: The output processing module primarily consists of shift accumulators and optional digital function blocks. One classical architecture of a shift accumulator is illustrated in Fig. 2. Two operations occur in parallel during each cycle: the adder tree generates a new partial sum, while the shift accumulator updates the accumulated result. As most DCIM architectures adopt MSB-first serial input streams, partial sums from consecutive cycles must be properly aligned for accumulation. Accordingly, the shift accumulator performs a one-bit left shift on the previously accumulated sum before adding the new partial sum, ensuring that MSB inputs are weighted appropriately, while LSB inputs are incorporated directly at the end of the input sequence. Optional digital function blocks may include sign-bit extension for signed/unsigned conversion, reconfigurable logic for adjusting weight precision, and global accumulators for computing aggregate sums across rows or tiles. These modules are typically composed of sub-blocks made up of identical cells to support parallel multibit processing. Preserving local regularity within these structures not only improves area efficiency but also enhances timing predictability and layout scalability.

One of the most notable classical digital in-memory computing architectures was presented by TSMC in 2021 [17]. In this design, the bitcell consists of a 6T SRAM cell combined with a 4T NOR cell. The adder tree features eight stages, enabling support for 256 parallel inputs from the bitcell array and utilizing modified systolic array adders. Another example was introduced by TSMC in 2022 [18], incorporating a 12T bitcell array and a six-stage adder tree optimized with full adders.

B. DCIM Component Design

1) Bitcell Array: In this work, the SRAM bitcell, as shown in Fig. 3, consists of a 6T SRAM cell combined with a 4T NOR cell. Each bank has a storage capacity of 128 bits, corresponding to a 16 × 8 bitcell array that supports 16-bit serial input as well as INT8 and INT4 operations for MAC computations. It is worth noting that the multiplication factor of each bitcell is preprocessed by a multibit-wide inverter prior to input, allowing the bitcell to be implemented with only ten transistors.

Unlike conventional SRAM arrays, the SRAM in this work is partitioned into multiple subrows, as further detailed in Section III-B, with each subrow serving as the first level of the adder tree and placed accordingly in the floorplan. To meet physical design and manufacturing constraints, the partitioned SRAM blocks are surrounded by dummy structures.

2) Peripheral Control Circuits: The correct read/write functionality of the circuit depends on the integration of peripheral circuits, which are similar to those used in conventional SRAMs. These typically include the following components.

1) Precharge circuit, which precharges the bitlines before read/write operations. This requires high current drive capability and is influenced by the parasitic parameters of the bitlines.
2) Row and column decoder circuits, which decode the read/write address information and convert it into the corresponding word line and bitline selector signals. These circuits typically include output driver units that could drive multiple gates and long metal interconnects.
3) Sense amplifiers, which are retained to provide a reliable data-reading interface, ensuring that the MAC operations remain synchronized with memory access.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:23 UTC from IEEE Xplore. Restrictions apply.

3) Reconfigurability: The circuit scales DCIM capacity by organizing storage into banks, with each bank containing 128 bits. All banks share a 16-bit-wide global serial data source for MAC input. In addition, a global output processing module is used to correctly map the computation results from different banks to the output.

Each bank supports two configuration modes: single-bank computation and dual-bank collaborative computation. As previously mentioned, the single-bank mode supports the highest INT8 computation precision. In the dual-bank collaborative mode, the high and low bits of the weight data are mapped to separate banks, thereby increasing the precision to INT16. To handle bit alignment in the computation results, a global shift accumulator between the two banks is activated.

In theory, by splitting and mapping the weight data, MAC operations can support any weight width that is a multiple of 8 bits. However, in most cases, a single bank is sufficient to meet the required precision.

4) BK Adder: The addition unit is the most area-intensive component of this design. Within the adder tree, each leaf node functions as an n-bit adder, which is also a critical element of both the shift accumulator and the output processing modules.

To improve computational speed, this work adopts the Brent–Kung (BK) adder in place of the conventional ripple-carry adder commonly used in typical DCIM designs. The BK adder reduces propagation delay in the adder tree and alleviates timing issues caused by long adder chains. It achieves this through a hierarchical carry propagation mechanism consisting of three stages: generate-and-propagate computation, recursive carry generation, and final sum computation. With a time complexity of $O(\log_2 N)$, the BK adder significantly outperforms ripple-carry adders, which exhibit linear delay. However, this performance gain comes at the cost of increased area, as the BK adder trades hardware efficiency for computational speed.

Structurally, the BK adder consists of three fundamental units: the propagate–generate (PG) unit, the dot operator, and the sum unit. Its tree-like dataflow offers high structural regularity. Nevertheless, very large computational bit-widths can result in an unbalanced aspect ratio, creating routing challenges. Moreover, for bit-widths exceeding 32 bits, the performance benefits diminish due to increased logic depth and an extended critical path, making it less suitable for large-scale operations.

In this work, both the adder tree for bitcell computation and the final shift accumulator are implemented using the BK adder. These components dominate the layout area and constitute a primary focus of the DAMIL-DCIM design.

C. Templated-Based DCIM Layout Generation

DCIM circuits feature a unique structure with memory-like components, necessitating a placement strategy that effectively leverages their architectural characteristics while properly managing parasitic-sensitive components. Despite the increasing interest in digital in-memory computing, research on DCIM layout automation remains limited. To date, one of the few notable works in this domain is AutoDCIM [3], which employs a template-based layout strategy. As the only published work on DCIM layout automation, it serves as a key reference for both this survey and our experimental comparisons.

AutoDCIM adopts a template-driven macro generation approach, using customized templates to guide overall circuit placement. In this framework, bitcells are organized within a single bank, with the adder tree placed adjacent to the bank. This simplifies the layout process by enforcing a predefined structure and reducing design complexity. However, it also imposes rigid constraints that limit adaptability across different technology nodes and circuit configurations. Given the absence of alternative methodologies, AutoDCIM serves as a critical benchmark for evaluating new approaches to DCIM layout automation.

In modern DCIM designs, however, the digital implementation of wide adder trees occupies significant area [19], leading to layout size mismatches with the bitcell bank. AutoDCIM addresses this challenge by introducing a bitcell active interleaving strategy. Its fundamental building block comprises SRAM-based weight cells and 1-bit multipliers arranged in an $m:1$ ratio. At any given time, only one weight cell is active for multiplication with a multiplier cell. As a result, only $1/m$ of the weight cells contribute to the computation simultaneously, substantially reducing the number of partial sums and narrowing the adder tree. While this approach helps minimize the size of the adder tree, it leads to inefficient utilization of circuit resources and significantly lowers throughput.

To mitigate layout-induced parasitics, AutoDCIM employs handcrafted layout templates that enable automatic layout synthesis while preserving functional correctness. However, the overly constrained nature of these templates limits their adaptability, resulting in suboptimal area utilization when applied to circuits with minor digital variations or across different technology nodes. Consequently, template-generated DCIM layouts can incur up to 60% area overhead compared to those generated by commercial P&R tools.

In parallel, a number of dataflow- and structure-aware physical design techniques have recently emerged. These methods incorporate bit-stack alignment, datapath-oriented constraints, and intrinsic dataflow optimizations to improve PPA metrics. The key insight behind these techniques is that adherence to dataflow guidance and structural regularity enhances circuit performance. Although originally developed for fully digital accelerators, these strategies are not directly applicable to DCIM placement. Nevertheless, they have inspired us to integrate dataflow- and structure-aware principles into the template-based framework, with the goal of improving the PPA metrics of DCIM circuits.

III. DAMIL-DCIM+ Framework

The overall flow of the DAMIL-DCIM framework is illustrated in Fig. 4. DAMIL-DCIM takes circuit features, component cell layouts, and user-specific design requirements as inputs and generates a DRC-clean placement for subsequent routing and verification. The process begins with applying a tree-structured, dataflow-based DCIM template to create a global floorplan, which captures the unique binary tree dataflow of DCIM and defines the relative locations of key components. Subsequently, an MILP-based module layout

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:23 UTC from IEEE Xplore. Restrictions apply.

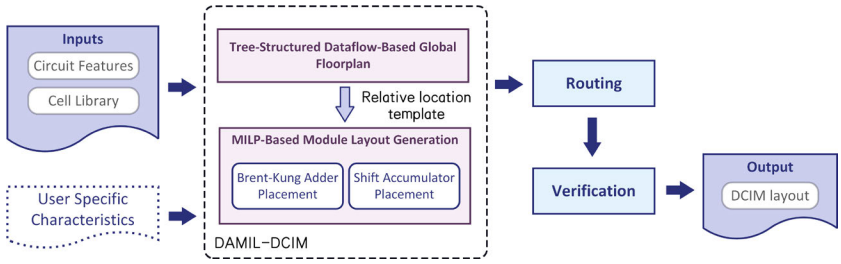


Fig. 4. Overview of the proposed DAMIL-DCIM+ flow.

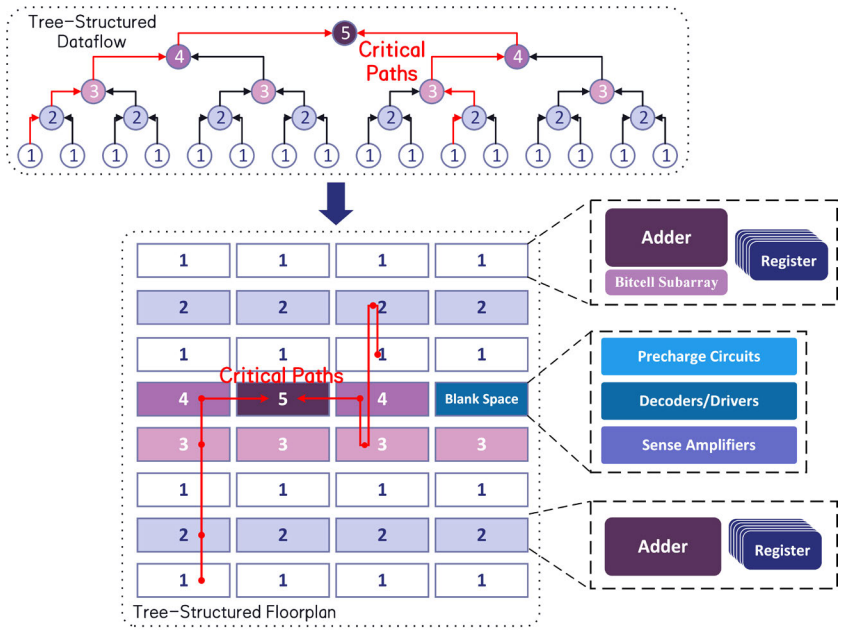


Fig. 5. Overview of the tree-structured dataflow-based global floorplan flow.

generation method is employed to produce the detailed placement of digital modules. For certain components, such as BK adders and shift accumulators, customized MILP constraints are introduced to accelerate solving while preserving local structural regularity.

A. Self-Assembling Design Methodology

This work uses the layouts of basic units, such as single-bit addition units and basic logic gates, as user inputs. These layouts are manually designed only once during the early design stage, making them a one-time effort that requires minimal time. Any user-defined layout with well-defined logical relationships and compliant constraints can also serve as system input. DAMIL-DCIM+ processes the netlist information and assembles these basic units into complex functional modules.

Digital logic layouts follow a structure similar to a digital standard cell, where the layout height must be an odd multiple of the power rail pitch. This ensures that each module, when placed within the power network, automatically satisfies pin connection requirements. The power rail connections in DAMIL-DCIM+ also follow standard digital backend practices, with power stripes connected at the top level.

For SRAM layouts, a self-assembly design methodology is employed, in which individual bitcells can be scaled into larger storage arrays through simple copying and translation. The size mismatch between the addition tree and the SRAM must also be considered. The layout design must preserve structural regularity while ensuring routing feasibility. Specifically, all SRAM array output pins must align with the input terminals of the first-level adders, as discussed in Section III-B. For peripheral circuits, self-assembly similarly emphasizes structural regularity based on bitlines. For example, sense amplifiers and precharge circuits connect to rows of SRAM cells via bitlines, which must be strictly vertically aligned to achieve optimal performance. Likewise, the output alignment of row decoders must precisely match the SRAM wordlines to ensure proper layout and connectivity.

B. Tree-Structured Dataflow-Based Global Floorplan

Since the adder tree occupies a significant portion of the area, DAMIL-DCIM+ adopts a novel tree-structured global layout strategy to efficiently integrate computation and memory. As illustrated in Fig. 5, the adder tree serves as the top-level organizational backbone of the layout, with its dataflow paths forming a binary tree topology. The figure contains two subfigures: the circular-node diagram represents the logical binary tree structure, whereas the rectangular-node diagram shows the corresponding optimized floorplan, where each rectangle represents the layout of one computational module. In this work, each *node* explicitly corresponds

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:23 UTC from IEEE Xplore. Restrictions apply.

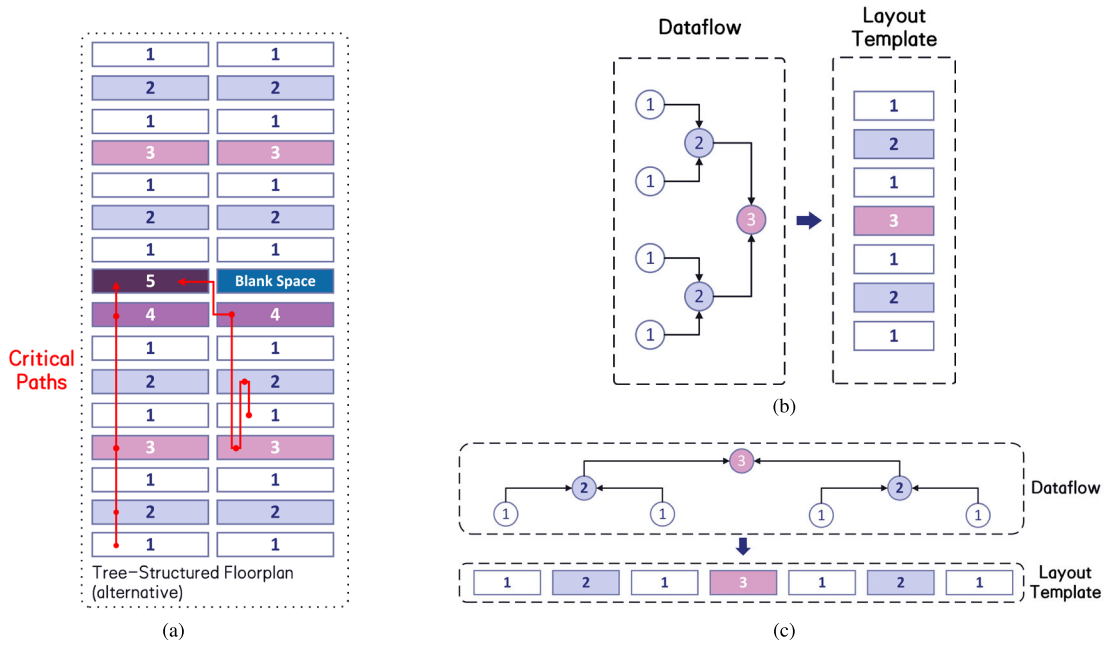


Fig. 6. Details in global floorplan template generation. (a) Template with two nodes per row. (b) Vertical floorplan template generation scheme. (c) Horizontal floorplan template generation scheme.

to one adder unit at a specific computation stage. The numerical index of each node (e.g., 1, 2, 3, 4, 5) denotes the level of the addition operation: node 1 units are directly connected to bitcell subarrays, while non-first-level nodes (nodes with index $>$ 1) represent adders that aggregate results from lower-level nodes. Each non-first-level adder unit typically consists of a multibit adder and an output register for pipeline segmentation.

Conventional SRAM arrays are tightly packed and typically occupy a smaller area than the adder tree in DCIM, resulting in significant area mismatches. These mismatches complicate floorplanning and routing and also introduce large differences in timing paths, creating additional challenges for meeting chip-level timing constraints. Inspired by [18], which proposed localizing bitcell banks near early-stage adders, DAMIL-DCIM+ extends this idea by integrating a bitcell subarray, an adder, and an optional output register within each first-stage node, while higher-stage nodes exclude memory components. To further alleviate area mismatches, global bitcell banks are partitioned into multiple local banks and co-located with their corresponding first-level adders (node 1). Each first-level node therefore integrates an adder, a local bitcell bank, and, if required, an output register, ensuring balanced area utilization.

Although the required bit width of the adder increases with each level, DAMIL-DCIM+ employs a unified adder module designed for the maximum local bit width across all levels. Consequently, all rectangular nodes in Fig. 5 have the same width and similar structure. This uniform sizing avoids alignment issues caused by heterogeneous adder widths and simplifies placement. In extreme cases, using a uniform adder may introduce about 10% additional area for the adder tree, but this overhead is offset by improved layout density and structural regularity.

This adder tree-centric layout improves critical-path alignment and global routability—two major challenges in DCIM, where large adder trees often conflict with dense bitcell banks. DAMIL-DCIM+ leverages the hierarchical structure of the dataflow graph to construct configurable binary tree templates. For example, the template in Fig. 5 arranges four nodes per row, whereas Fig. 6(a) shows an alternative configuration with two nodes per row, both derived from the same five-level adder tree. In Fig. 6(a)–(c), the layout recursively expands both vertically and horizontally (as shown in Fig. 6(b) and (c), equalizing path lengths from leaf nodes to the root to balance signal propagation. This recursive structure reduces routing congestion and improves interconnect efficiency.

Finally, peripheral circuits and control logic, such as decoders and precharge units, are placed within predefined blank regions of the template to maintain layout regularity and enable compact integration. The output processing module is located at the top-level node to optimize signal propagation. While the reserved regions for peripheral circuits may introduce slight irregularities compared with computational nodes, such deviations are acceptable. Through joint optimization of memory–compute integration and signal alignment, DAMIL-DCIM+ achieves a structure-driven, dataflow-aware floorplan that maximizes layout efficiency while minimizing unused area and routing overhead.

C. MILP-Based Module Layout Generation

The tree-structured floorplan discussed in Section III-B provides a global placement framework; however, a dedicated detailed placement method is still required for certain modules. The MILP approach is particularly well-suited for block-level detailed placement because it guarantees globally optimal solutions while satisfying all design constraints.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:23 UTC from IEEE Xplore. Restrictions apply.

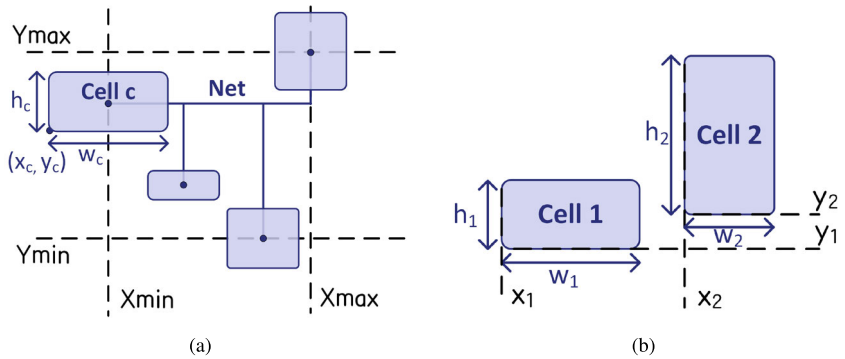


Fig. 7. Basic MILP placement formulation. (a) HPWL formulation in MILP style. (b) Overlap constraint formulation in MILP style.

After obtaining the global floorplan, DAMIL-DCIM+ employs an MILP-based approach to further optimize the internal layout of individual modules. We apply this MILP-based module layout generation method to two key components: the BK adder and the shift accumulator, which are critical to both timing and area efficiency. By explicitly capturing placement constraints such as cell alignment, routing channels, and pin accessibility, the MILP formulation enables the automated generation of compact and timing-aware layouts for these modules. To accelerate the inherently time-consuming MILP process for larger designs while preserving local structural regularity, we introduce customized constraints tailored to the architectures of these two modules.

Although our discussion focuses on the BK adder and shift accumulator, the proposed MILP formulation is general and can be readily extended to other types of adders or similar arithmetic circuits by incorporating their structural constraints into the same optimization framework, demonstrating its broad applicability to digital module placement.

1) Basic MILP Placement Formulation: The MILP formulation for the basic placement problem is formulated in the following equation:

$$\begin{aligned} &\textbf{Minimize} \sum_{n\in\text{Nets}} \text{HPWL}_n \\ &\textbf{Subject to } \text{overlap}(c_i, c_j) = 0 \\ &\qquad \text{out Of Bound}(c_i) = 0 \\ &\qquad \forall c_i, c_j \in \text{Cells}, \quad c_i \neq c_j. \end{aligned} \tag{1}$$

The placement problem in VLSI design primarily aims to minimize the total wire length while ensuring that modules are non-overlapping and constrained within the designated layout area. Since the exact wire length cannot be determined prior to detailed placement and routing, the half-perimeter wire length (HPWL) is widely adopted as an efficient and practical estimation metric during early design stages.

HPWL is formulated in a linear form in (2), where $X\text{max}_n$, $Y\text{max}_n$, $X\text{min}_n$, and $Y\text{min}_n$ represent the upper and lower x and y boundaries of the centers of each cell connected by net n, as illustrated in Fig. 7(a). Here, x_c, y_c, w_c, and h_c represent the x-coordinate and y-coordinate of the bottom-left corner, width and height of cell c, respectively,

$$\begin{aligned} &\text{HPWL}_n = X\text{max}_n + Y\text{max}_n - X\text{min}_n - Y\text{min}_n \\ &X\text{max}_n \geq x_c + w_c/2, \quad Y\text{max}_n \geq y_c + h_c/2 \\ &X\text{min}_n \leq x_c + w_c/2, \quad Y\text{min}_n \leq y_c + h_c/2 \\ &\qquad \forall c \in n \ \forall n \in \text{Nets}. \end{aligned} \tag{2}$$

To formulate non-overlap constraints in a linear form, four binary auxiliary variables are introduced to represent the relative positions between each pair of cells. As illustrated in (3), these variables indicate whether one cell is to the left, right, above, or below the other. To enforce the non-overlap condition, it is sufficient for exactly one of these variables to be true, thereby specifying a valid directional separation between the two cells.

The corresponding linear constraints are constructed using a sufficiently large constant M, which ensures that the inequality becomes active only when the associated auxiliary variable is true. Otherwise, the constraint is trivially satisfied and does not affect the solution space.

An illustrative example is provided in Fig. 7(b), where Cell 1 is placed to the left of Cell 2. In this case, the auxiliary variable aux_{left} is set to true, while all other directional variables are set to false

$$\begin{aligned} &x_i + w_i \leq x_j + M \cdot (1 - \text{aux}_{\text{left}}) \\ &x_j + w_j \leq x_i + M \cdot (1 - \text{aux}_{\text{right}}) \\ &y_i + h_i \leq y_j + M \cdot (1 - \text{aux}_{\text{below}}) \\ &y_j + h_j \leq y_i + M \cdot (1 - \text{aux}_{\text{above}}) \\ &\text{aux}_{\text{left}} + \text{aux}_{\text{right}} + \text{aux}_{\text{above}} + \text{aux}_{\text{below}} = 1 \\ &\text{aux}_{\text{left}}, \text{aux}_{\text{right}}, \text{aux}_{\text{above}}, \text{aux}_{\text{below}} \in \{0, 1\} \\ &\qquad \forall i, j \in \text{Cells}, \quad i \neq j \\ &M \text{ is sufficiently large.} \end{aligned} \tag{3}$$

The boundary constraints are defined in (4), where X_{min}, Y_{min}, X_{max}, and Y_{max} specify the permissible ranges of the $x-$ and y-coordinates for each cell. Typically, X_{min} and Y_{min} are set to zero, defining the origin of the placement region. This formulation represents the most common type of boundary constraint, though it can be adjusted to accommodate specific placement requirements

$$\begin{aligned} &x_i \geq X_{\text{min}}, \quad x_i + w_i \leq X_{\text{max}} \\ &y_i \geq Y_{\text{min}}, \quad y_i + h_i \leq Y_{\text{max}} \quad \forall i \in \text{Cells}. \end{aligned} \tag{4}$$

2) Customized BK Adder MILP Placement: The BK adder features a unique tree-like dataflow consisting of three basic functional cells: the unit cells of *pg*, *dot*, and *sum*. Fig. 8(a) shows the typical dataflow of a 12-bit BK adder.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:23 UTC from IEEE Xplore. Restrictions apply.

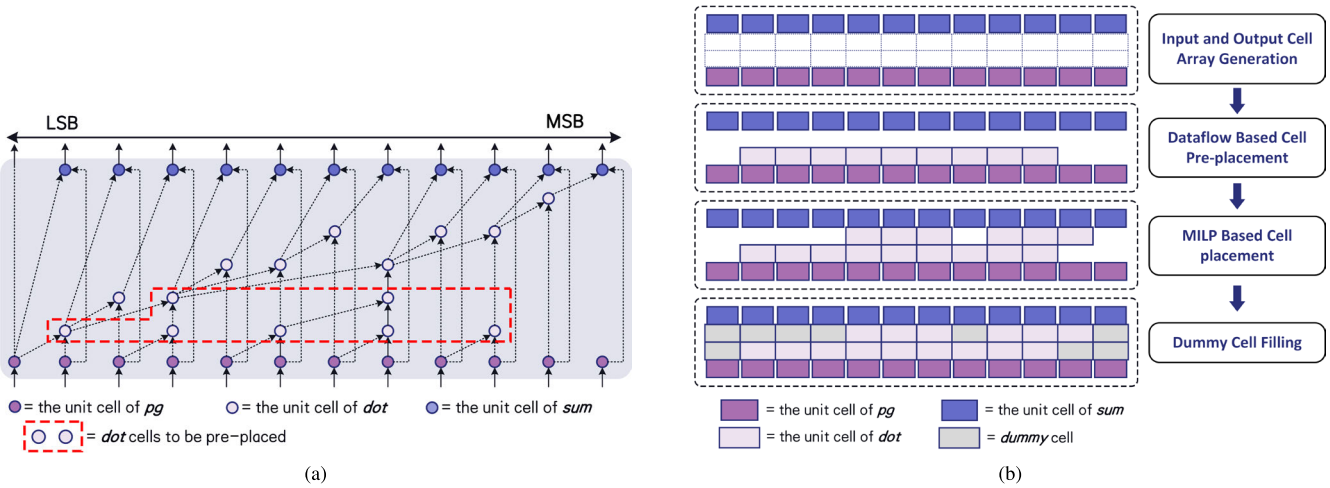


Fig. 8. BK adder detailed placement strategy. (a) Typical BK adder dataflow. (b) Customized BK adder placement flow.

The placement process is divided into four steps to optimize layout regularity and align dataflow, as illustrated in Fig. 8(b).

1) *Input and output cell array generation:* We generate unit cells for *pg* and *sum*, and align them vertically on a bit-by-bit basis. For adders with large input bit widths, the input and output cell arrays are folded to achieve the desired layout aspect ratio. A virtual grid for *dot* cells is then generated based on the array width, the dimensions of a single *dot* cell, and the total number of required dot cells.
   The *pg* and *sum* generation cells correspond directly to individual input and output bits, respectively, allowing for the natural formation of array structures. Therefore, these two types of cells are vertically aligned. When the input bit width becomes large, folding is applied to the cell arrays to maintain a compact and practical layout aspect ratio. This virtual grid for *dot* operation cells serves as the basis for subsequent placement steps.
2) *Dataflow-based cell preplacement:* We apply a regular digital placement of *dot* cells to obtain a preplacement, as indicated by the red dashed box in Fig. 8(a). The resulting cell positions are subsequently used to formulate relative position constraints in the MILP model. These constraints empirically accelerate the MILP solving process by more than 10×.
   To further speed up the ILP-based placement in the next step, we experimentally preplace *dot* operation cells following a clear and regular dataflow structure, as shown in the same figure. Cells directly connected to the input PG cells are positioned near their corresponding inputs, while cells dependent on these preplaced units are placed between them.
   While this strategy may introduce minor increases in local wirelength, we evaluated the HPWL of the final layout with and without the preplacement step. Results indicate that the increase in local HPWL is under 6%, while achieving over a 10× reduction in MILP runtime—demonstrating a favorable trade-off between layout quality and optimization efficiency.
3) *MILP-Based Cell Placement:* We complete the placement of the remaining *dot* cells using the MILP formulation described in Section III-C1. In this step, the placement problem is modeled as an MILP, where the coordinates of each cell are constrained to be integer multiples of the grid dimensions defined in the first step. Specifically, the location of each cell is represented as a pair of integers (x_g, y_g), such that the actual coordinates are given by $x = x_g \cdot \text{grid}_w$ and $y = y_g \cdot \text{grid}_h$, where grid_w and grid_h represent the grid's width and height, respectively. Additional constraints ensure that all dot operation cells are placed within the designated layout region.
4) *Dummy cell filling:* After all the cells are placed, some grid locations may remain unoccupied after placement. To ensure consistent placement density and maintain manufacturability, corresponding dummy cells are inserted into these vacant grid positions. This uniform filling not only preserves layout regularity but also enhances the reliability of the circuit fabrication process.

3) Customized Shift Accumulator MILP Placement: DAMIL-DCIM+ introduces array constraints for shift accumulator submodules. Shift accumulators generally include submodules with multiple identical cells for parallel multi-bit processing. Additional array constraints enforce local regularity, improving signal flow and speeding up the MILP solving process. For multibit cell arrays, various shapes with different row and column counts can be selected. The shape is automatically determined within the MILP placement formulation and is represented by a row and column pair (r, c). The set of potential shapes, denoted as Shapes, can be easily determined based on the bit width. For each multibit cell array submodule, we introduce binary auxiliary variables corresponding to the number of possible array shapes |Shapes|. Only one of these binary variables is true, representing the selected shape of the cell array. The actual width and height of the submodule are then calculated, as formulated in (5). Fig. 9 illustrates an example of four potential configurations of a 6-bit inverter array, with shapes of 1r6c (one row by six columns),

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:23 UTC from IEEE Xplore. Restrictions apply.

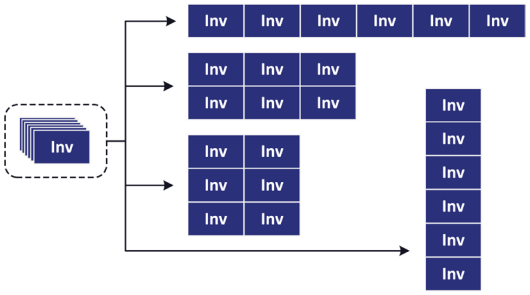


Fig. 9. Example of potential array configurations.

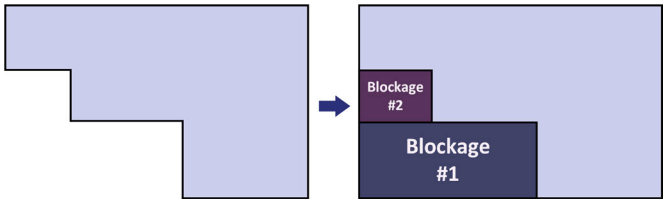


Fig. 10. Modeling a non-rectangular placement boundary with virtual blockages for MILP formulation.

2r3c, 3r2c, and 6r1c, respectively,

$$w_{\text{submodule}} = \sum_{i=1}^{|\text{shapes}|} \text{aux}_i \cdot w_{\text{cell}} \cdot r_i$$

$$h_{\text{submodule}} = \sum_{i=1}^{|\text{shapes}|} \text{aux}_i \cdot h_{\text{cell}} \cdot c_i$$

$$\sum_{i=1}^{|\text{Shapes}|} \text{aux}_i = 1$$

$$\text{aux}_i \in \{0, 1\}, \quad 1 \le i \le |\text{Shapes}|. \tag{5}$$

The global floorplanning process, while aiming for a compact layout, may produce non-rectangular placement regions for certain subsidiary modules. For example, the area allocated for the accumulator block, located above the adder tree, may be defined as a rectilinear polygon to accommodate variations in adjacent peripheral circuits.

To handle such irregular placement regions within our MILP formulation, we adopt a modeling technique based on virtual blockages. First, a simple rectangular bounding box is defined to fully enclose the non-rectangular region. Virtual rectangular blockages are then inserted into this bounding box to represent areas where cells cannot be placed. Standard non-overlap constraints are applied between these blockages and all placeable cells, effectively restricting placement to the valid irregularly shaped region. Fig. 10 illustrates this approach, showing how two virtual blockages are used to model the placement boundary.

IV. Experimental Results

All experiments were conducted on a Linux workstation with up to 16 Intel 3.0 GHz CPU cores allocated to our process. The primary runtime bottleneck in our framework is the MILP solver. For large-scale modules, such as 32-bit adders and accumulators, the solver can require significant computation time to find a globally optimal solution. However, we observe that the solution quality improves marginally after an initial optimization period of a few hours.

To manage computation time effectively, we implement an early termination strategy that ensures a practical and predictable turnaround time. A 12-h time limit is enforced for each module's placement, with optimizers for individual modules executed in parallel. This approach caps the maximum runtime for the entire DCIM placement process at 48 h.

The runtime limit is aligned with common physical design schedules. Methodologies that rely on manually crafted templates or extensive placement constraints for commercial tools typically involve an iterative design process over several days. As such, the 48-h maximum runtime of the DAMIL-DCIM+ flow is designed to fit within the typical timeframe of a single design cycle.

Empirically, DAMIL-DCIM+ outperforms baseline methods within a limited runtime budget. While some modules may not achieve the global optimum, potentially leading to minor increases in local wire length, our results in Section IV-A demonstrate that our framework still significantly reduces wire length compared to existing approaches.

Fig. 11 presents the completed layout generated by the DAMIL-DCIM+ framework, with a core area of $1.132 \times 0.580 = 0.657$ mm^2. The layout reveals that the adder tree occupies the majority of the core, while the peripheral circuits and control logic of the bitcell bank are placed in the remaining available space, guided by the tree-structured global floorplan. To optimize die area utilization, the two accumulators are positioned above the adder tree, leveraging a segmented horizontal boundary.

A. Comparisons With Other Placement Methods

As discussed in Sections I and II, several placement frameworks have been developed in previous research. To comprehensively evaluate the effectiveness of our proposed framework, we implemented a complete physical design flow for the DCIM circuit using various placement strategies. Specifically, we compare DAMIL-DCIM+ against the following three placement approaches.

1) *Innovus:* We compare with placement in a commercial physical tool, Cadence Innovus, a widely used digital physical implementation platform in the IC industry, which serves as a baseline placement method. Its automated placement flow optimizes the design for reduced wire length and improved timing performance.
2) *Template-based method:* We compare our approach with the template-based AutoDCIM method. Since the original implementation is unavailable, we reproduced their methodology to serve as a representative template-based placement approach.
3) *Innovus with SDP:* Structured datapath (SDP) refers to a circuit architecture with a repetitive and regular dataflow, commonly used in arithmetic and data processing units such as adders, multipliers, MAC units, and shift registers.

In a standard P&R flow, traditional standard cell placement is primarily optimized based on logical connections in the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:23 UTC from IEEE Xplore. Restrictions apply.

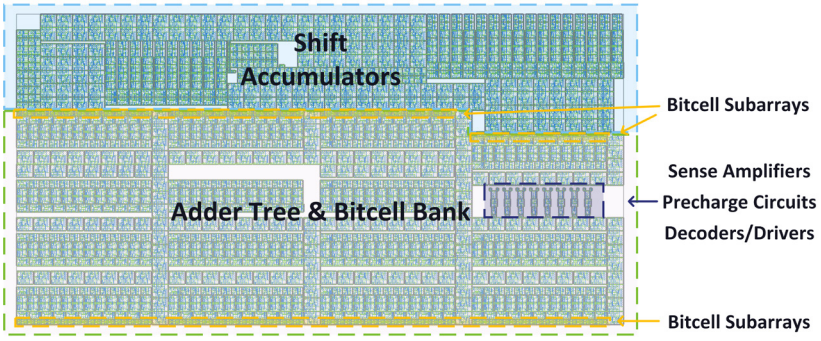


Fig. 11. Top-level layout of the practical DCIM circuit.

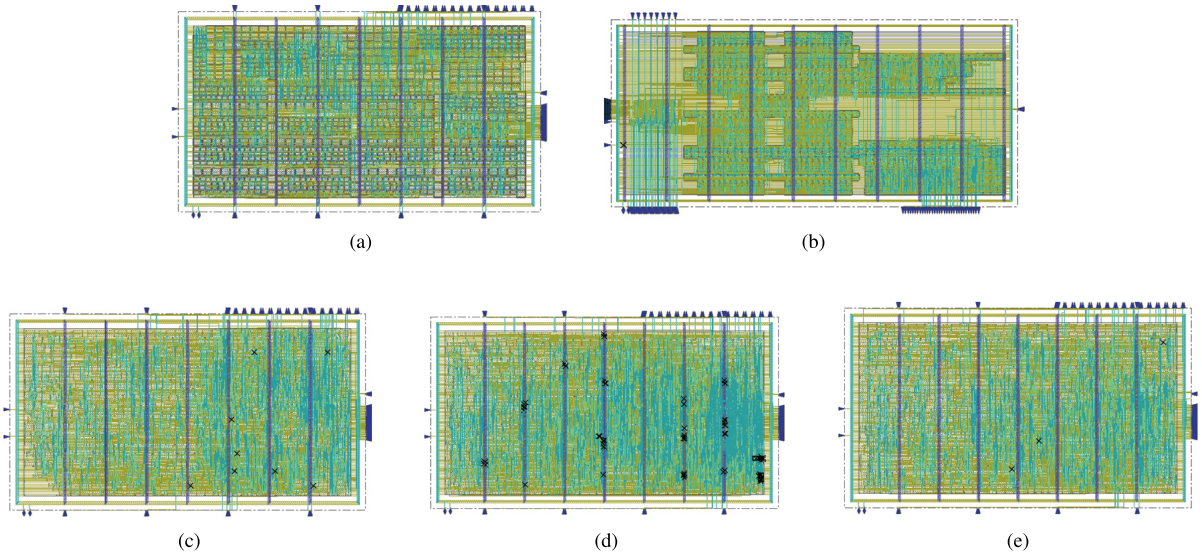


Fig. 12. Results of different placement frameworks after routing. (a) DAMIL-DCIM+. (b) Template-based method. (c) Innovus. (d) Innovus with SDP. (e) Innovus with SDP*(* Run Innovus SDP flow with larger target area to avoid placement failure. Details given in Section IV-A).

netlist. However, if not properly managed, SDP circuits may lead to the following issues.

1) Disrupt dataflow regularity, leading to suboptimal cell distribution.
2) Introduce long interconnections, increasing timing delays and power consumption.
3) Cause routing congestion, degrading overall performance.

Cadence Innovus addresses these issues by optimizing SDP circuits to ensure.

1) *Logical cell alignment:* Aligns datapath cells along rows or columns to minimize wire length.
2) *Minimized critical path length:* Reduces key interconnects to enhance timing.
3) *Reduced routing congestion:* Enforces structured layouts for regular routing patterns, improving routability.
4) *Optimized power consumption:* Prevents excessive dynamic power due to long wires.

We compare our approach with the SDP placement strategy in Cadence Innovus, where bitlines in the bitcell bank are designated as SDPs. Innovus-SDP enforces a regular structure to optimize critical paths, automatically managing related cells and arranging them according to the critical dataflow. This serves as a representative of emerging dataflow-aware placement methods.

After obtaining the placement results, we perform routing and complete the physical design flow using Cadence Innovus, ensuring an identical workflow for each approach. The target clock period is set to 3.6 ns. For the Innovus Placement Method and Innovus-SDP Placement Method, a target core area and aspect ratio must be specified before placement. To ensure a fair comparison, we set these parameters to match the layout generated by our framework. The Template-Based Method, however, does not require a predefined area target and automatically determines the layout. While Innovus-SDP attempts to align related bitline cells, its strict constraints significantly limit the placement engine's optimization flexibility. This often leads to legalization failures and suboptimal placement, resulting in excessively long routing wires, increased parasitics, and degraded timing and power performance. To mitigate these failures, we re-ran the Innovus-SDP flow with a larger core area, denoted as Innovus with SDP.

After completing the full design flow, the result of which is shown in Fig. 12, and we evaluate the post-layout circuits by measuring total wire length, power consumption, and timing metrics. The PPA score, shown in the last row of Table I, is computed as the product of maximum power, equivalent

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:23 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON WITH OTHER RELATED PLACEMENT FRAMEWORK

	Innovus	Ratio	Template-Based Method [3]	Ratio	Innovus with SDP	Ratio	Innovus with SDP*	Ratio	DAMIL-DCIM	Ratio
Core Area (mm^2)	**0.657**	**1.00**	1.052	1.60	**0.657**	**1.00**	0.685	1.04	**0.657**	**1.00**
Total Wire Length (mm)	495.377	1.00	**294.842**	**0.60**	823.771	1.66	548.285	1.11	310.02	0.63
Max Power (mW)	6.539	1.00	**3.951**	**0.60**	11.702	1.79	8.088	1.24	5.256	0.80
WNS (ns)	-0.798	-	0.079	**-**	-7.316	**-**	-1.184	**-**	**0.088**	**-**
TNS (ns)	-12.174	-	**0.000**	**-**	-216.127	-	-5.031	**-**	**0.000**	**-**
Equivalent Period (ns)	4.398	1.00	3.521	0.80	10.916	2.48	4.784	1.09	**3.512**	**0.80**
PPA ($mW \times ns \times mm^2$)	18.894	1.00	14.634	0.77	83.925	4.44	26.505	1.40	**12.128**	**0.64**

$*$ Run Innovus SDP flow with larger target area to avoid placement failure. Details in Section IV-A.

TABLE II
MACRO SPECIFICATION SUMMARY

Specification	Value
Process Node	28 nm
Array Size	128 bits
Macro Area	1.052 mm^2
Supply Voltage	1.05 V
Input Bit-width	8 (sequential input)
Weight Bit-width	8
Output Bit-width	32
Parallel Inputs	16
Throughput	0.88G MAC/s
Energy Efficiency	27.8 TOPS/W

TABLE III
CLOCK TREE QUALITY COMPARISON WITH INNOVUS

	Innovus	DAMIL-DCIM
Min ID (ns)	-0.471	**-0.553**
Max ID (ns)	0.017	**-0.416**
Avg ID (ns)	-0.329	**-0.493**
Std.Dev. ID (ns)	0.100	**0.019**
Skew (ns)	0.488	**0.137**

period, and core area. Table I compares the resulting layouts of DAMIL-DCIM+ and the baseline methods. The results indicate that DAMIL-DCIM+ achieves near-optimal total wire length and timing metrics while obtaining the highest PPA score among all methods.

Although the main focus of this work lies in optimizing the layout quality rather than maximizing ultimate circuit-level performance, we include additional circuit-level metrics to provide more context for readers in the CIM community. Specifically, Table II summarizes key specifications of the post-layout macro, including the supply voltage (1.05 V), input/output/weight precision (8/32/8 bits), array size (128 bits), and supported parallel inputs (16). To further enhance interpretability, we also report commonly used CIM-specific metrics such as throughput (0.88G MAC/s) and energy efficiency (27.8 TOPS/W). While these metrics are not the primary optimization targets of our method, they serve to demonstrate that the resulting layouts still maintain competitive efficiency under a reasonable configuration.

Experimental results demonstrate that our method effectively reduces area overhead compared to template-based approaches. While the template-based method achieves slightly better power consumption and total wire length, it results in a core area of $1.570 \times 0.670 = 1.0519\,mm^2$, which is 1.60× larger with its standard cell density of only 56.05%. This sparse layout is impractical and significantly degrades the final PPA score. The reduced total parasitic capacitance in the template-based layout explains its improved timing and power consumption; however, this benefit comes at the cost of inefficient area utilization and potential manufacturability concerns.

All placements were processed using an identical routing script and DRC-fixing flow in Cadence Innovus when routing. Despite this uniform routing methodology, only the layout produced by DAMIL-DCIM+ achieved a DRC-clean result (Fig. 12). The high placement density and unstructured nature of the layouts from competing methods resulted in significant routing pressure, which the tool could not resolve even after attempting DRC corrections. This outcome demonstrates that our dataflow-aware placement methodology creates a fundamentally more routable layout, preemptively mitigating congestion and enabling high-quality layout implementation.

Compared to Innovus, DAMIL-DCIM+ significantly reduces total wire length and improves timing, validating the effectiveness of our global floorplanning strategy and MILP-based detailed placement.

These timing improvements stem from lower parasitics due to shorter wires and the optimized placement of registers within the adder tree, which enhances clock tree quality. Table III presents Insertion Delay (ID) and skew data for the clock tree in the setup-late timing corner. The results demonstrate that DAMIL-DCIM+ achieves lower and more tightly distributed IDs, reducing skew and directly enhancing timing performance. Furthermore, while SDP placement (Innovus-SDP) attempts to optimize critical paths through dataflow alignment, our DAMIL-DCIM+ framework outperforms it in terms of wire length, power efficiency, and timing metrics, further confirming the superiority of our placement strategy.

B. Post-Layout Verification on Parasitic-Sensitive Paths

To evaluate the robustness of DAMIL-DCIM+ under parasitic effects, post-layout simulations were performed for both local bitcell operations and critical long interconnect paths.

Fig. 13 illustrates a single-bitcell read/write operation. During the first half of the simulation, a write operation stores a high value in the bitcell located at the first row and first column. In the second half, a read operation successfully retrieves the stored value. The red dashed boxes highlight

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:23 UTC from IEEE Xplore. Restrictions apply.

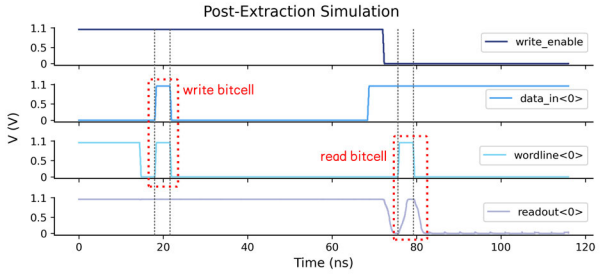


Fig. 13. Post-layout bitcell read/write simulation.

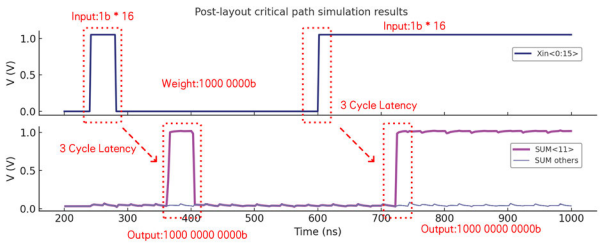


Fig. 14. Post-layout critical path simulation with accumulator.

these transitions, confirming that the proposed design reliably handles local parasitic-sensitive bitline paths.

To further evaluate functionality under more parasitic-sensitive conditions, we extended the analysis to a BK adder tree, which involves longer routing and higher parasitic loading. In this experiment, a 16×8 bitcell array was programmed with 16 `1000 0000b` operands. The input signal `Xin` initiates computation through a three-stage pipeline, and the output result appears after three clock cycles. The intended operation is given by

$$\texttt{1000 0000b} \times \texttt{1b} \times 16 = \texttt{1000 0000 0000b}.$$

As shown in Fig. 14, the transient simulation output correctly asserts `SUM<11>`, while all other outputs remain low, thereby confirming the correct functionality of the BK adder tree and demonstrating that the proposed framework maintains robust performance even in parasitic-sensitive long-path scenarios.

V. Conclusion

In this work, we introduce DAMIL-DCIM+, a dedicated placement framework tailored to the unique requirements of DCIM circuits. Our approach combines traditional template-based techniques with a global dataflow-aware floorplanning strategy, while leveraging MILP formulations for fine-grained placement optimization. We evaluate the effectiveness of DAMIL-DCIM+ on a practical DCIM circuit, demonstrating that it not only achieves superior PPA metrics but also effectively manages parasitic-sensitive paths—significantly outperforming state-of-the-art placement methods.

References

[1] C.-J. Jhang, C.-X. Xue, J.-M. Hung, F.-C. Chang, and M.-F. Chang, "Challenges and trends of SRAM-based computing-in-memory for AI edge devices," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 5, pp. 1773–1786, May 2021.

[2] M. R. Guthaus, J. E. Stine, S. Ataei, B. Chen, B. Wu, and M. Sarwar, "OpenRAM: An open-source memory compiler," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2016, pp. 1–6.

[3] J. Chen et al., "AutoDCIM: An automated digital CIM compiler," in *Proc. 60th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2023, pp. 1–6.

[4] M. Liu, X. Tang, K. Zhu, H. Chen, N. Sun, and D. Z. Pan, "OpenSAR: An open source automated end-to-end SAR ADC compiler," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Nov. 2021, pp. 1–9.

[5] E. Chang, "BAG2: A process-portable framework for generator-based ams circuit design," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Apr. 2018, pp. 1–8.

[6] Z. He, P. Liao, S. Liu, Y. Ma, Y. Lin, and B. Yu, "Physical synthesis for advanced neural network processors," in *Proc. 26th Asia South Pacific Design Autom. Conf.*, Jan. 2021, pp. 833–840, doi: 10.1145/3394885.3431625.

[7] T. T. Ye and G. D. Micheli, "Data path placement with regularity," in *IEEE/ACM Int. Conf. Comput. Aided Design. (ICCAD), IEEE/ACM Dig. Tech. Papers*, Nov. 2000, pp. 264–270.

[8] S. I. Ward et al., "Structure-aware placement techniques for designs with datapaths," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 32, no. 2, pp. 228–241, Feb. 2013.

[9] S. Chou, M.-K. Hsu, and Y.-W. Chang, "Structure-aware placement for datapath-intensive circuit designs," in *Proc. 49th Annu. Design Autom. Conf.*, Jun. 2012, pp. 762–767, doi: 10.1145/2228360.2228498.

[10] S. Bae, H.-O. Kim, J. Choi, and J. Park, "Coarse-grained structural placement for a synthesized parallel multiplier," in *Proc. Symp. Int. Symp. Phys. Design*, Mar. 2015, pp. 17–24, doi: 10.1145/2717764.2717775.

[11] Y. Wang and H. Shin, "Effective regularity extraction and placement techniques for datapath-intensive circuits," *IET Circuits, Devices Syst.*, vol. 11, no. 5, pp. 512–519, Sep. 2017.

[12] J. Zhang, W. Zhang, G. Luo, X. Wei, Y. Liang, and J. Cong, "Frequency improvement of systolic array-based CNNs on FPGAs," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2019, pp. 1–4.

[13] J.-M. Lin, W.-F. Huang, Y.-C. Chen, Y.-T. Wang, and P.-W. Wang, "DAPA: A dataflow-aware analytical placement algorithm for modern mixed-size circuit designs," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Nov. 2021, pp. 1–8.

[14] Y. Chou, J.-W. Hsu, Y.-W. Chang, and T.-C. Chen, "VLSI structure-aware placement for convolutional neural network accelerator units," in *Proc. 58th ACM/IEEE Design Autom. Conf. (DAC)*, Dec. 2021, pp. 1117–1122.

[15] D. Fang, B. Zhang, H. Hu, W. Li, B. Yuan, and J. Hu, "Global placement exploiting soft 2D regularity," in *Proc. Int. Symp. Phys. Design*, 2022, pp. 203–210, doi: 10.1145/3505170.3506723.

[16] H. Hu, D. Fang, W. Li, B. Yuan, and J. Hu, "Systolic array placement on FPGAs," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–9.

[17] Y.-D. Chih et al., "16.4 An 89TOPS/W and 16.3TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 252–254.

[18] H. Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm^2 fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[19] J.-S. Seo, "Advances and trends on on-chip compute-in-memory macros and accelerators," in *Proc. 60th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2023, pp. 1–6.



Chuyu Wang received the B.S. degree in microelectronics from Fudan University, Shanghai, China, in 2022, where he is currently pursuing the Ph.D. degree with the State Key Laboratory of Integrated Chips and Systems, Microelectronics Department.

His current research interests include global routing and CIM physical design automation.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:23 UTC from IEEE Xplore. Restrictions apply.



Xinglong Yan received the B.S. degree in microelectronic science and engineering from Tongji University, Shanghai, China, in 2024. He is currently pursuing the M.S. degree with the State Key Laboratory of Integrated Chips and Systems, Fudan University, Shanghai, China.

His current research interests include mixed signal design and analog design automation.



Zecheng Xu received the B.S. degree in microelectronics from Fudan University, Shanghai, China, in 2024, where he is currently pursuing the Ph.D. degree with the State Key Laboratory of Integrated Chips and Systems, Microelectronics Department.

His current research interests include global routing and analog-integrated circuit design automation.



Ke Hu received the B.S. degree from the School of Microelectronics, Fudan University, Shanghai, China, in 2022, where she is currently pursuing the Ph.D. degree.

Her research interests include sparse matrix processing, LU decomposition, and stochastic computing.



Keren Zhu (Member, IEEE) received the B.S. degree in electrical engineering with highest distinction from the University of Wisconsin-Madison, Madison, WI, USA, in 2016, and the Ph.D. degree from Chandra Family Department of Electrical and Computer Engineering, The University of Texas at Austin, Austin, TX, USA, in 2022.

He is currently an Assistant Professor with the School of Microelectronics, Fudan University, Shanghai, China. His current research interests include physical design automation, analog integrated circuit design automation, and logic synthesis.



Shuo Li (Member, IEEE) received the B.Eng. degree from the University of Electronic Science and Technology of China, Chengdu, China, in 2013, the M.S. degree from Fudan University, Shanghai, China, in 2016, both in microelectronics, and the Ph.D. degree in electrical engineering from the University of Virginia, Charlottesville, VA, USA, in 2021.

He was a Postdoctoral Associate at the University of Illinois at Urbana-Champaign, Urbana, IL, USA, and Yale University, New Haven, CT, USA, from 2021 to 2024. In August 2024, he joined the Frontier Institute of Chip and System, Fudan University, where he is currently an Assistant Professor. His research interests include analog/digital/mixed-signal integrated circuits and systems, energy harvesting and power management units, compute-in-memory accelerators, and low-power Internet-of-Things systems-on-chip.

Dr. Li was a recipient of the IEEE International Symposium on Circuits and Systems (ISCAS) Best Paper Award in 2020.



Fan Yang (Member, IEEE) received the B.S. degree from Xi'an Jiaotong University, Xi'an, China, in 2003, and the Ph.D. degree from Fudan University, Shanghai, China, in 2008.

He is currently a Full Professor with the School of Microelectronics, Fudan University. His research interests include model order reduction, circuit simulation, high-level synthesis, acceleration of artificial neural networks, and yield analysis and design for manufacturability.



Xuan Zeng (Senior Member, IEEE) received the B.S. and Ph.D. degrees in electrical engineering from Fudan University, Shanghai, China, in 1991 and 1997, respectively.

She is currently a Full Professor with the School of Microelectronics, Fudan University, where she served as the Director of the State Key Laboratory of Application Specific Integrated Circuits (ASIC) and Systems, from 2008 to 2012. She was a Visiting Professor with the Department of Electrical Engineering, Texas A&M University, College Station, TX, USA, and the Microelectronics Department, Technische Universiteit Delft, Delft, The Netherlands, in 2002 and 2003, respectively. Her current research interests include analog circuit modeling and synthesis, design for manufacturability, high-speed interconnect analysis and optimization, and circuit simulation.

Prof. Zeng received the Changjiang Distinguished Professor with the Ministry of Education Department of China in 2014, the Chinese National Science Funds for Distinguished Young Scientists in 2011, the First-Class of Natural Science Prize of Shanghai in 2012, the 10th For Women in Science Award in China in 2013, and the Shanghai Municipal Natural Science Peony Award in 2014. She received the Best Paper Award from the 8th IEEE Annual Ubiquitous Computing, Electronics and Mobile Communication Conference 2017.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:23 UTC from IEEE Xplore. Restrictions apply.